

6.1 Parallel and Serial Communication

Data transmission refers to the process of transferring data between two or more digital devices. Data is transmitted from one device to another in analog or digital format. Basically, data transmission enables devices or components within devices to speak to each other.

Data is transferred in the form of bits between two or more digital devices. There are two methods used to transmit data between digital devices: serial transmission and parallel transmission. Serial data transmission sends data bits one after another over a single channel. Parallel data transmission sends multiple data bits at the same time over multiple channels.

Serial transmission is normally used for long-distance data transfer. It is also used in cases where the amount of data being sent is relatively small. It ensures that data integrity is maintained as it transmits the data bits in a specific order, one after another. In this way, data bits are received in sync with one another.

Serial Communication

Introduction

Serial communication is common method of transmitting data between a computer and a peripheral device such as a programmable instrument or even another computer. Serial communication transmits data one bit at a time, sequentially, over a single communication line to a receiver. Serial is also a most popular communication protocol that is used by many devices for instrumentation. This method is used when data transfer rates are very low or the data must be transferred over long distances and also where the cost of cable and synchronization difficulties makes parallel communication impractical. Serial communication is popular because most computers have one or more serial ports, so no extra hardware is needed other than a cable to connect the instrument to the computer or two computers together.

Serial and Parallel Transmission

Let us now try to have a comparative study on parallel and serial communications to understand the differences and advantages & disadvantages of both in detail.

We know that parallel ports are typically used to connect a PC to a printer and are rarely used for other connections. A parallel port sends and receives data eight bits at a time over eight separate wires or lines. This allows data to be transferred very quickly. However, the setup looks more bulky because of the number of individual wires it must contain. But, in the case of a serial communication, as stated earlier, a serial port sends and receives data, one bit at a time over one wire. While it takes eight times as long to transfer each byte of data this way, only a few wires are required. Although this is slower than parallel communication, which allows the transmission of an entire byte at once, it is simpler and can be used over longer distances. So, at first sight it would seem that a serial link must be inferior to a parallel one, because it can transmit less data on each clock tick. However, it is often the case that, in modern technology, serial links can be clocked considerably faster than parallel links, and achieves a higher data rate.

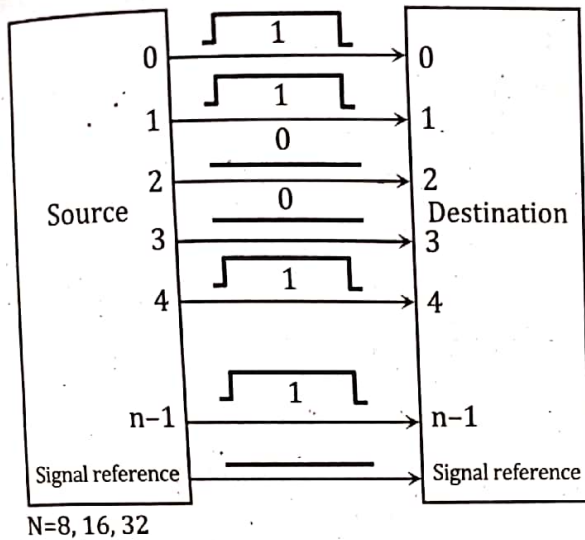


Figure 6.1: Parallel Transmission

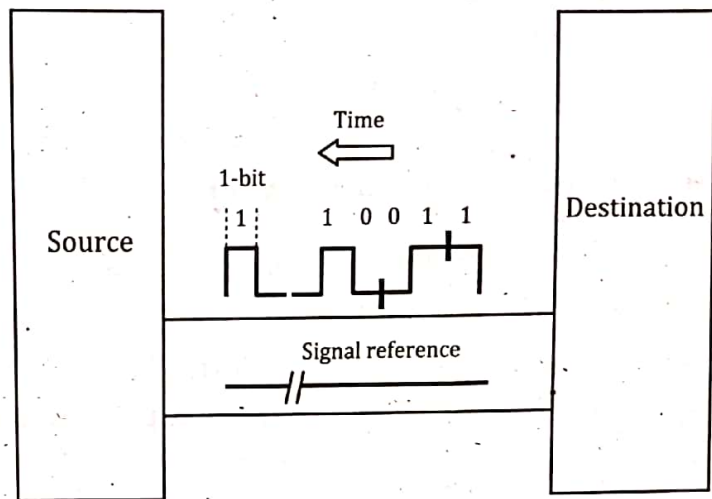


Figure 6.2: Serial Transmission

Even in shorter distance communications, serial computer buses are becoming more common because of a tipping point where the disadvantages of parallel buses (clock skew, interconnect density) outweigh their advantage of simplicity. The serial port on your PC is a full duplex device meaning that it can send and receive data at the same time. In order to be able to do this, it uses separate lines for transmitting and receiving data.

From the above discussion we could understand that serial communications have many advantages over parallel one like:

- ⌘ Requires fewer interconnecting cables and hence occupies less space.
- ⌘ "Cross talk" is less of an issue, because there are fewer conductors compared to that of parallel communication cables.
- ⌘ Many ICs and peripheral devices have serial interfaces.
- ⌘ Clock skew between different channels is not an issue.
- ⌘ Cheaper to implement.

Clock Skew

Clock skew is a phenomenon in synchronous circuits in which the clock signal sent from the clock circuit arrives at different components at different times, which can be caused by many things, like:

- ⌘ Wire-interconnect length
- ⌘ Temperature variations
- ⌘ Variation in intermediate devices
- ⌘ capacitive coupling
- ⌘ Material imperfections

Serial transmission is normally used for long-distance data transfer. It is also used in cases where the amount of data being sent is relatively small. It ensures that data integrity is maintained as it transmits the data bits in a specific order, one after another. In this way, data bits are received in-sync with one another.

Advantages and Disadvantages of Using Parallel Data Transmission

The main advantages of parallel transmission over serial transmission are:

- ⌘ It is easier to program;
- ⌘ and data is sent faster.

Although parallel transmission can transfer data faster, it requires more transmission channels than serial transmission. This means that data bits can be out of sync, depending on transfer distance and how fast each bit loads. A simple example of where this can be seen is with a voice over IP (VOIP) call when distortion or interference is noticeable. It can also be seen when there is skipping or interference on a video stream.

Parallel transmission is used when

- ⌘ a large amount of data is being sent;
- ⌘ the data being sent is time-sensitive;
- ⌘ and the data needs to be sent quickly.

A scenario where parallel transmission is used to send data is video streaming. When a video is streamed to a viewer, bits need to be received quickly to prevent a video pausing or buffering. Video streaming also requires the transmission of large volumes of data. The data being sent is also time-sensitive as slow data streams result in poor viewer experience.

Serial Data Transmission Modes

When data is transmitted between two pieces of equipment, three communication modes of operation can be used.

Simplex: In a simple connection, data is transmitted in one direction only. For example, from a computer to printer that cannot send status signals back to the computer.

Half-duplex: In a half-duplex connection, two-way transfer of data is possible, but only in one direction at a time.

Full duplex: In a full-duplex configuration, both ends can send and receive data simultaneously, which technique is common in our PCs.

Serial Data Transfer Schemes

Like any data transfer methods, Serial Communication also requires coordination between the sender and receiver. For example, when to start the transmission and when to end it, when one particular bit or byte ends and another begins, when the receiver's capacity has been exceeded, and so on. Here comes the need for synchronization between the sender and the receiver. A protocol defines the specific methods of coordinating transmission between a sender and receiver. For example a serial data signal between two PCs must have individual bits and bytes that the receiving PC can distinguish. If it doesn't, then the receiving PC can't tell where one byte ends and the next one begins or where one bit ends and begins. So the signal must be synchronized in such a way that the receiver can distinguish the bits and bytes as the transmitter intends them to be distinguished. There are two ways to synchronize the two ends of the communication.

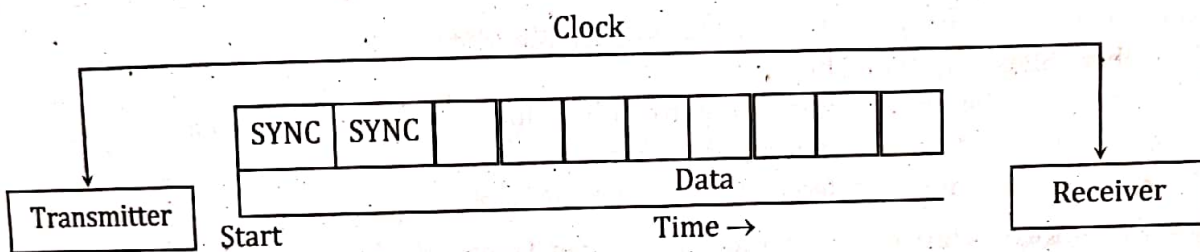
1. Synchronous data transmission
2. Asynchronous data transmission

Synchronous Data Transmission

The synchronous signaling methods use two different signals. A pulse on one signal line indicates when another bit of information is ready on the other signal line.

In synchronous transmission, the stream of data to be transferred is encoded and sent on one line, and a periodic pulse of voltage which is often called the "clock" is put on another line, that tells the receiver about the beginning and the ending of each bit.

- ⌘ A more efficient method of transferring serial data is to synchronize the transmitter and the receiver and then send a large block of data characters one after the other with no time between characters.
- ⌘ No start or stop bits are needed with individual data characters because the receiver automatically knows that every 8 bits received after synchronization represents a data character.
- ⌘ To indicate start of transmission, transmitter sends out one or more unique characters called sync. Characters.
- ⌘ The receiver uses the sync characters or the flag to synchronize its internal clock with that of the receiver.
- ⌘ ISDN, High Speed Modems and Digital Communication Channel use synchronous transmission.



(a) Synchronous serial transmission format.

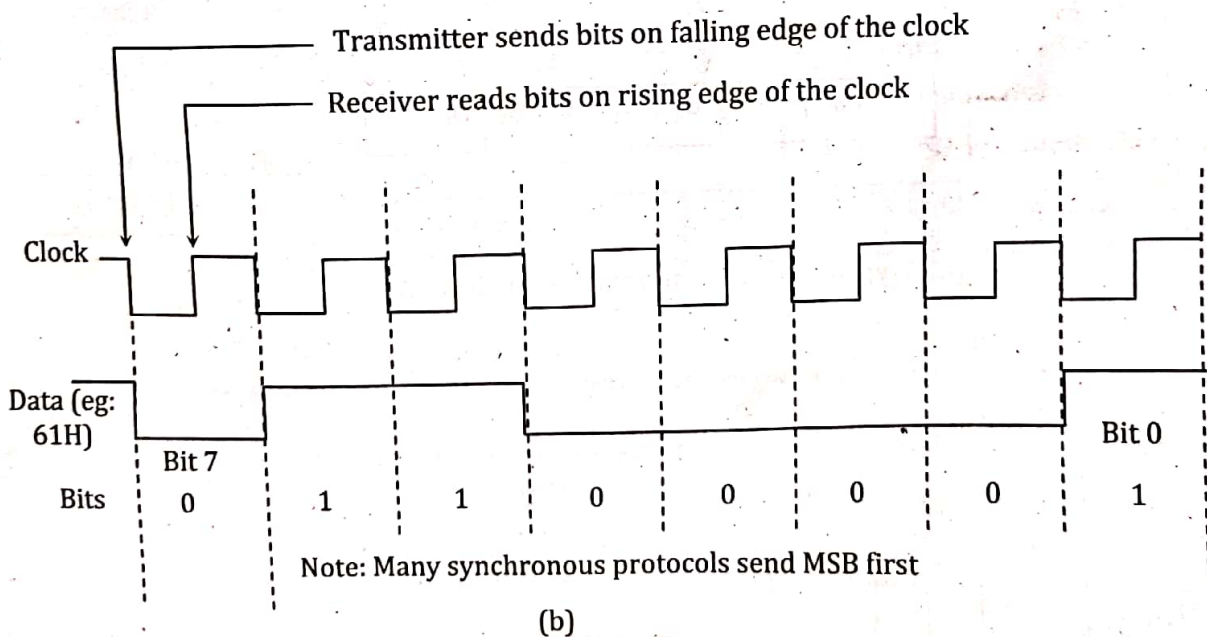


Figure 6.3: Synchronous transmission

Advantages: The only advantage of synchronous data transfer is the Lower overhead and thus, greater throughput, compared to asynchronous one.

Disadvantages

- ⌘ Slightly more complex
- ⌘ Hardware is more expensive

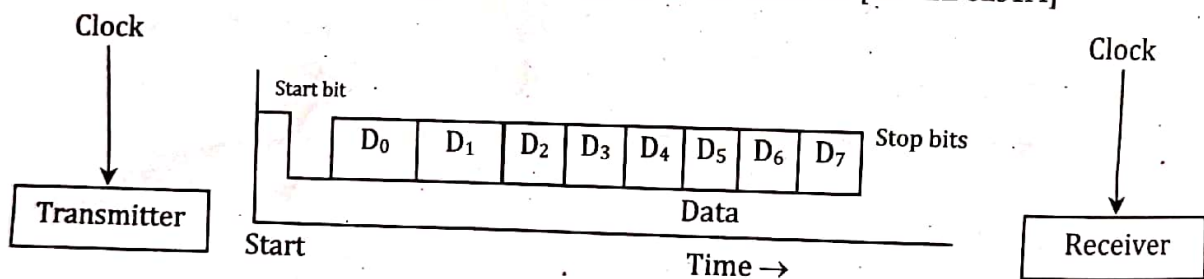
Asynchronous Data Transmission

The asynchronous signaling methods uses only one signal. The receiver uses transitions on that signal to figure out the transmitter bit rate (known as auto baud), and timing. A pulse from the local clock indicates when another bit is ready. That means synchronous transmissions use an external clock, while asynchronous transmissions uses special signals along the transmission medium. Asynchronous communication is the commonly prevailing communication method in the personal computer industry, due to the reason that it is easier to implement and has the unique advantage that bytes can be sent whenever they are ready, no need to wait for blocks of data to accumulate.

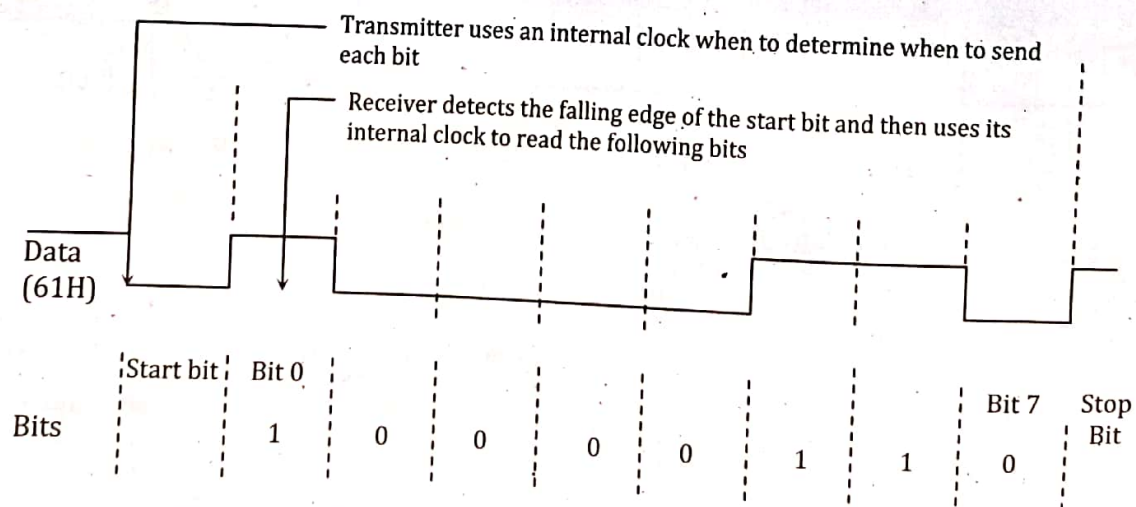
- ⌘ For asynchronous transmission, each data character has a bit which identifies its start and 1 or 2 bits which identify end.
- ⌘ Since each character is individually identified, character can be sent at any time asynchronously.
- ⌘ The beginning of a data character is identified by the line going low for 1 bit time. This bit is called start bit.
- ⌘ The data bits are then sent out on the line one after the other.
- ⌘ Note that LSB is transmitted first.
- * After parity bit, the signal line is returned high for at least 1 bit time to identify the end of the character. This high bit is always referred as stop bit.

UART: Universal Asynchronous Receiver Transmitter [IN 8250]

USART: Universal Synchronous Asynchronous Receiver Transmitter [INTEL 8251A]



(a) Asynchronous serial transmission format.



Note: Asynchronous protocols send LSB first
(b)

Figure 6.4: Asynchronous Transmission

Advantages A1

- ⌘ Simple and doesn't require much synchronization on both communication sides.
- ⌘ The timing is not as critical as for synchronous transmission; therefore hardware can be made cheaper.
- ⌘ Set-up is very fast, so well suited for applications where messages are generated at irregular intervals, for example data entry from the keyboard.

Disadvantages A2

One of the main disadvantages of asynchronous technique is the large relative overhead, where a high proportion of the transmitted bits are uniquely for control purposes and thus carry no useful information.

Key Differences between Serial and Parallel Transmission A-1

- ⌘ Serial transmission requires a single line to communicate and transfer data whereas, parallel transmission requires multiple lines.
- ⌘ Serial transmission used for long distance communication whereas, the parallel transmission used for shorter distance.
- ⌘ Error and noise are least in serial as compared to parallel transmission. Since one bit follows another in Serial Transmission whereas, in Parallel Transmission multiple bits are sent together.
- ⌘ Parallel transmission is faster as the data is transmitted using multiples lines whereas, in Serial transmission data flows through a single wire.
- ⌘ Serial Transmission is full duplex as the sender can send as well as receive the data whereas, Parallel Transmission is half duplex since the data is either sent or received.
- ⌘ Serial transmission cables are thinner, longer and economical in comparison with the Parallel Transmission cables.
- ⌘ Serial Transmission is reliable and straightforward whereas, Parallel Transmission is unreliable and complicated.

Conclusion

Serial - Full duplex

Parallel - Half duplex (data either sent or received).

Both Serial and Parallel Transmission have their advantages and disadvantages respectively. Parallel Transmission is used for shorter distance, provides greater speed. On the other hand, Serial Transmission is reliable for transferring data to longer distance. Hence, we conclude that both serial and parallel are individually essential for transferring data.

8251 Universal Synchronous Asynchronous Receiver Transmitter

The 8251 is a USART (Universal Synchronous Asynchronous Receiver Transmitter) for serial data communication. As a peripheral device of a microcomputer system, the 8251 receives parallel data from the CPU and transmits serial data after conversion. This device also receives serial data from the outside and transmits parallel data to the CPU after conversion.

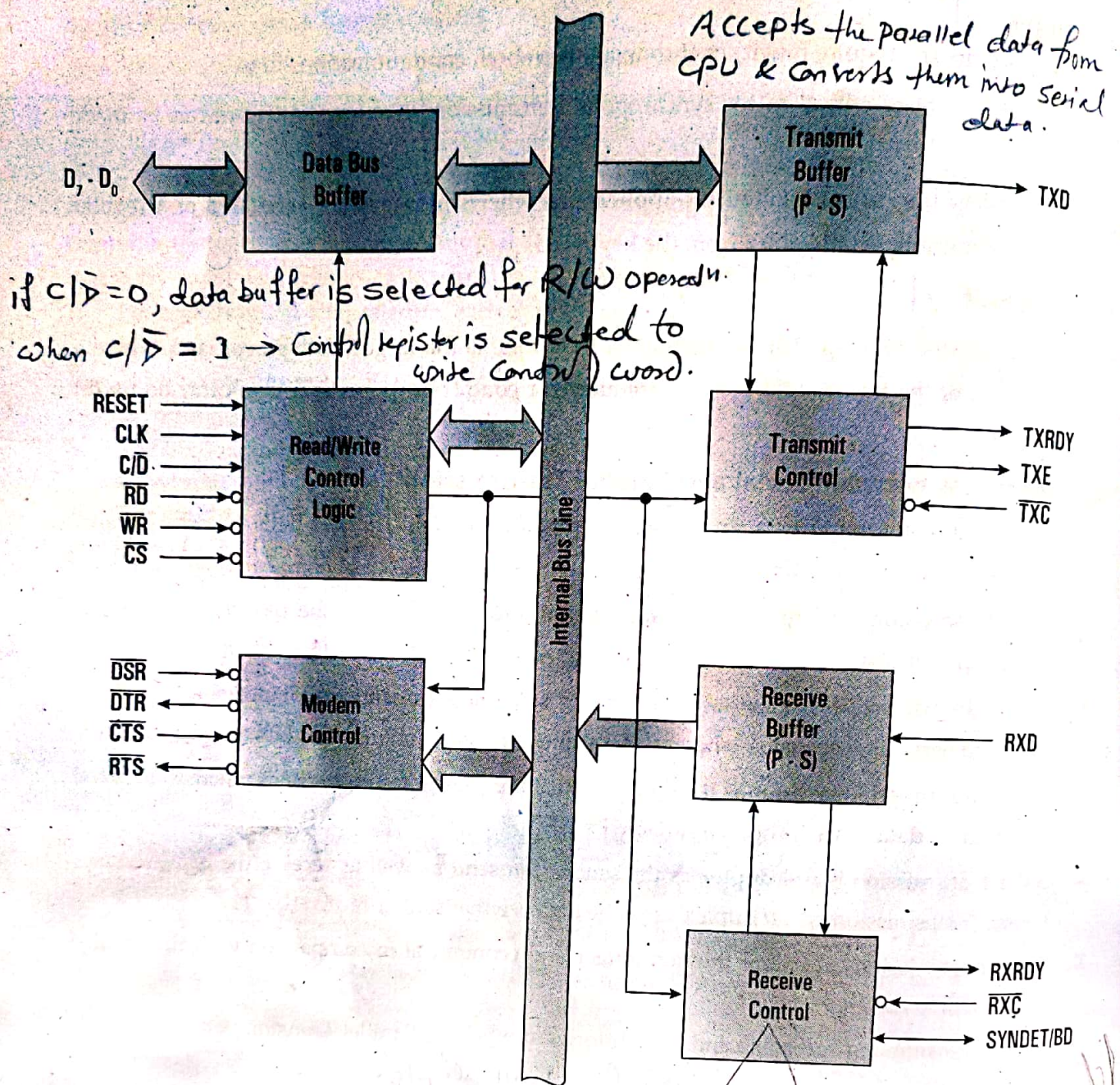


Figure 6.5: Block diagram of 8251 USART

The 8251 functional configuration is programmed by software. Operation between the 8251 and a CPU is executed by program control. Table 1 shows the operation between a CPU and the device.

$\overline{CS}$	$C/\bar{D}$	$\overline{RD}$	$\overline{WR}$	
1	x	x	x	Data Bus 3-State
0	x	1	1	Data Bus 3-State
0	1	0	1	Status $\rightarrow$ CPU
0	1	1	0	Control Word $\leftarrow$ CPU
0	0	0	1	Data $\rightarrow$ CPU
0	0	1	0	Data $\leftarrow$ CPU

Table 6.1: Control signal for read and write operation

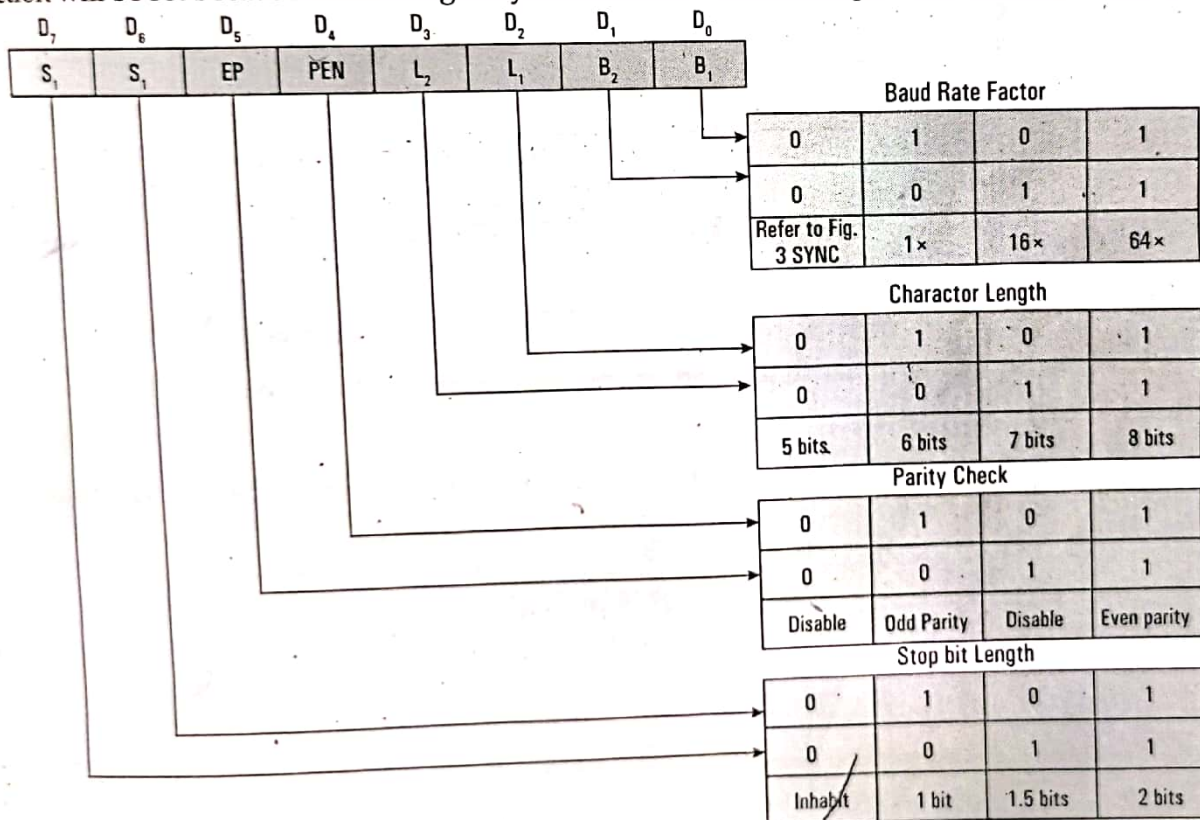
Control Words

1. Mode Instruction

Mode instruction is used for setting the function of the 8251. Mode instruction will be in "wait for write" at either internal reset or external reset. That is, the writing of a control word after resetting will be recognized as a "mode instruction." Items set by mode instruction are as follows:

- ⌘ Synchronous/ asynchronous mode
- ⌘ Stop bit length (asynchronous mode)
- ⌘ Character length
- ⌘ Parity bit
- ⌘ Baud rate factor (asynchronous mode)
- ⌘ Internal/ external synchronization (synchronous mode)
- ⌘ Number of synchronous characters (Synchronous mode)

The bit configuration of mode instruction is shown in Figures 2 and 3. In the case of synchronous mode, it is necessary to write one or two byte sync characters. If sync characters were written, a function will be set because the writing of sync characters constitutes part of mode instruction.

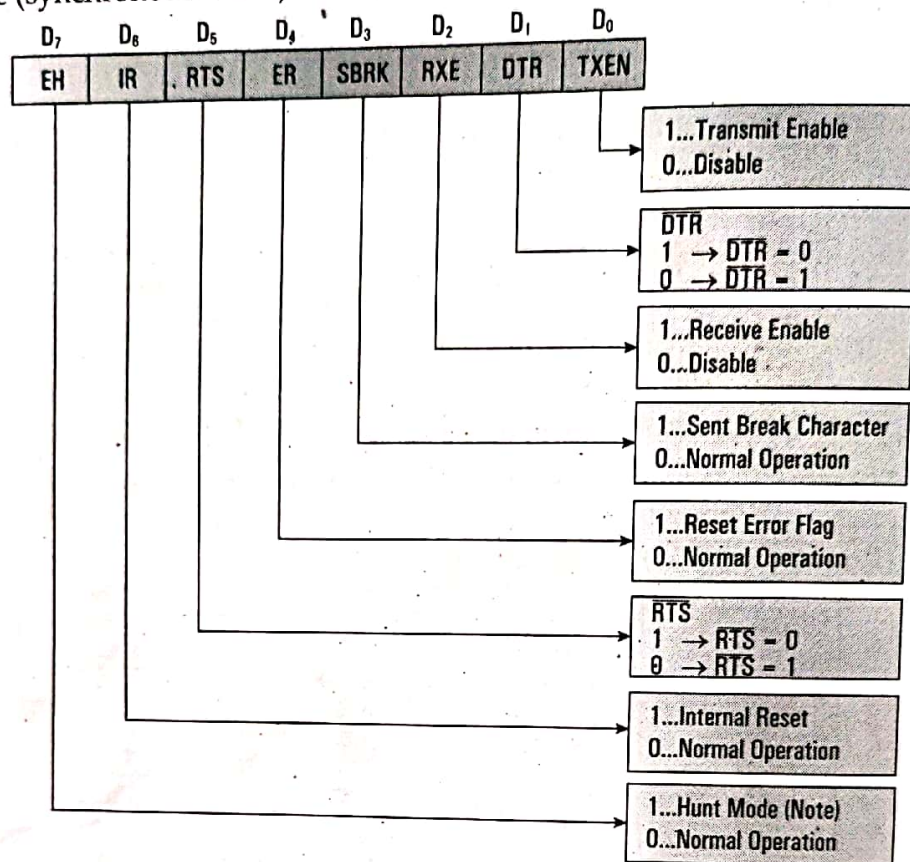


2. Command Word

Command is used for setting the operation of the 8251. It is possible to write a command whenever necessary after writing a mode instruction and sync characters. Items to be set by command are as follows:

- ⌘ Transmit Enable/ Disable
- ⌘ Receive Enable/ Disable
- ⌘ DTR, RTS Output of data
- ⌘ Resetting of error flag

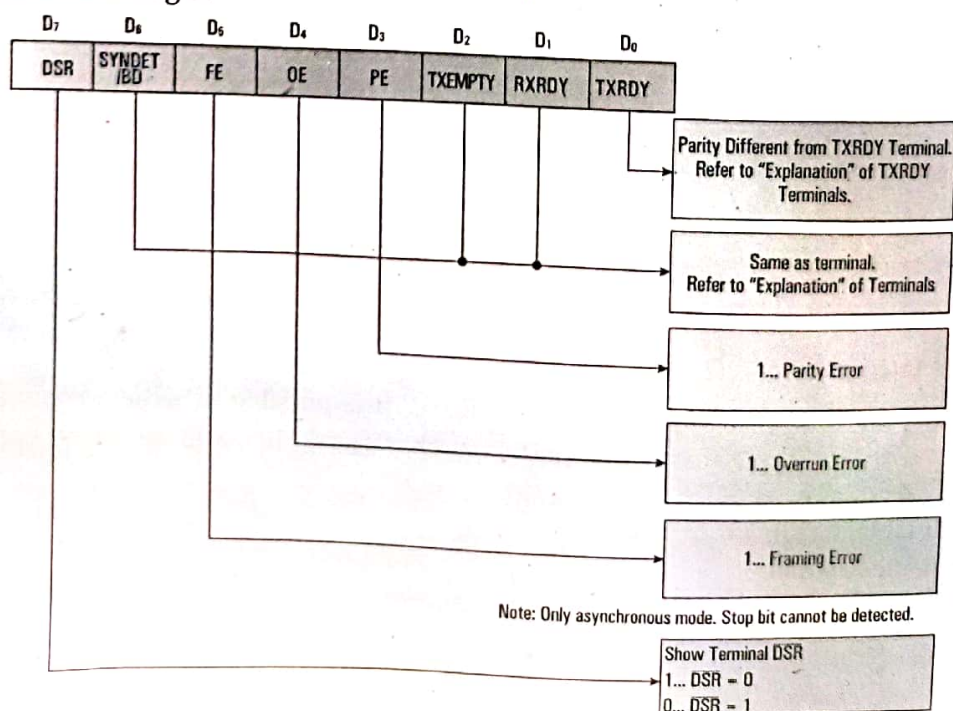
- ⌘ Sending to break characters
- ⌘ Internal resetting
- ⌘ Hunt mode (synchronous mode)



Note: Search mode for synchronous characters in synchronous mode.

3. Status Word

It is possible to see the internal status of the 8251 by reading a status word. The bit configuration of status word is shown in Fig 5.



Note: Only asynchronous mode. Stop bit cannot be detected.

Pin Description

D_0 to D_7 [I/O Terminal]

This is bidirectional data bus which receives control words and transmits its data from the CPU and sends status words and received data to CPU.

RESET [Input Terminal]

A "high" on this input forces the 8251 into "reset status". The device waits for writing of "mode instruction". The minimum reset width is six clock inputs during the operating status of CLK.

CLK [Input Terminal]

CLK signal is used to generate internal device timing. CLK signal is independent of RXC or TXC. However, the frequency of CLK must be greater than 30 times the RXC and TXC at Synchronous mode and Asynchronous "x1" mode, and must be greater than 5 times at Asynchronous "x16" and "x64" mode.

WR [Input Terminal]

This is the "active low" terminal which receives a signal for writing transmitted data and control words from the CPU into the 8251.

RD [Input Terminal]

This is the "active low" terminal which receives a signal for reading transmitted data and control words from the CPU into the 8251.

C/D [Input Terminal]

This is an input terminal which receives a signal for selecting data or command words and status words when the 8251 is accessed by the CPU.

If C/D=low, data will be accessed. If C/D=high, command word or status word will be accessed.

CS [Input Terminal]

This is the "active low" input terminal which selects the 8251 at low level when the CPU accesses.

Note: The device won't be in "standby status"; only setting CS = High.

TXD [Output Terminal]

This is an output terminal for transmitting data from which serial-converted data is sent out. The device is in "mark status" (high level) after resetting or during a status when transmit is disabled. It is also possible to set the device in "break status" (low level) by a command.

TXRDY [Output Terminal]

This is an output terminal which indicates that the 8251 is ready to accept a transmitted data character. But the terminal is always at low level if CTS=high or the device was set in "TX status" by a command.

Note: TXRDY status word indicates disable and that transmit data character is receivable of CTS or command.

TXEMPTY [Output Terminal]

This is an output terminal which indicates that the 8251 has transmitted all the characters and had no data character. In "synchronous mode" the terminal is at high level, if transmitted data are no longer remaining and sync characters are automatically transmitted. If the CPU writes a data character, TXEMPTY will be reset by the leading edge of WR signal. Note: As the transmitter is disabled by setting CTS "High" or command, data written before disable will be sent out. Then TXD and

TXEMPTY will be "High". Even if a data is written after disable, that data is not sent out and TXE will be "High". After the transmitter is enabled, it sent out.

TXC [Input Terminal]

This is a clock input signal which determines the transfer speed of transmitted data. In "synchronous mode," the baud rate will be the same as the frequency of TXC. In "asynchronous mode", it is possible to select the baud rate factor by mode instruction. It can be 1, 1/16 or 1/64 the TXC. The falling edge of TXC shifts the serial data out of the 8251.

RXD [Input Terminal]

This is a terminal which receives serial data.

RXRDY [Output Terminal]

This is a terminal which indicates that the 8251 contains a character that is ready to READ. If the CPU reads a data character, RXRDY will be reset by the leading edge of RD signal. Unless the CPU reads a data character before the next one is received completely, the preceding data will be lost. In such a case, an overrun error flag status word will be set.

RXC [Input Terminal]

This is a clock input signal which determines the transfer speed of received data. In RXC "synchronous mode" the baud rate is the same as the frequency of "asynchronous mode", it is possible to select the baud rate factor by mode instruction. It can be 1, 1/16, 1/64 the RXC.

SYNDET/ BD [Input or Output Terminal]

This is a terminal whose function changes according to mode. In "internal synchronous mode", this terminal is at high level, if sync characters are received and synchronized. If a status word is read, the terminal will be reset. In "external synchronous mode", this is an input terminal. A "High" on this input forces the 8251 to start receiving data characters.

In "asynchronous mode," this is an output terminal which generates "high level" output upon the detection of a "break" character if receiver data contains a "low-level" space between the stop bits of two continuous characters. The terminal will be reset, if RXD is at high level.

DSR [Input Terminal]

This is an input port for MODEM interface. The input status of the terminal can be recognized by the CPU reading status words.

DTR [Output Terminal]

This is an output port for MODEM interface. It is possible to set the status of DTR by a command.

CTS [Input Terminal]

This is an input terminal for MODEM interface which is used for controlling a transmit circuit. The terminal controls data transmission if the device is set in "TX Enable" status by a command. Data is transmittable if the terminal is at low level.

RTS [Output Terminal]

This is an output port for MODEM interface. It is possible to set the status RTS by a command.

Programming 8251A

- ⌘ According to the datasheet, the 8251A requires a worst case recovery time of 16 cycles.
- ⌘ The initialization sequence of 8251A is hence lengthy.
- ⌘ The second reason is that, the 8251A does not respond correctly to a hardware reset.

- ✖ This means that you have to delay 16 processor clock cycles on power up.
- ✖ Therefore a series of software command must be sent to the device to the 8251A distinguishes a command word from a mode word by the order in which make sure it reset properly before the desired mode and command word are sent.
- ✖ They are sent to the device
- ✖ Any words sent to the command address after the mode word will be treated as command word until the device is reset.

Techniques

A simple way to produce the required delay and a margin of safety is to load CX with 2 and count it down with the loop instruction

LOOP First > 1 Clock Cycles
LOOP Last > 5 Clock Cycles

=====

MOV DX 0FFF2H • Command Register Address of 8251 A

6.2 Methods of Parallel Data Transfer

1. Simple I/O

- ⌘ When you need to get digital data from a simple switch, all you have to do is connect the switch to an I/P port line and read the value.
- ⌘ Likewise, when you need to O/P data to simple LED, all you have to do is connect the LED to an O/P port and send the value.
- ⌘ The LED is always ready, so you can send data at any time.

→ The data is always present and ready so that it can be read at any time.

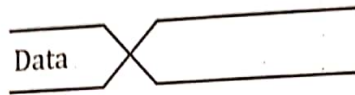
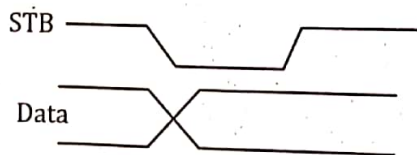


Figure 6.6: Timing diagram of simple I/O

2. Strobe I/O : A strobe is a signal that is sent

- ⌘ In many applications, valid data is present on an external device only at a certain time, so it must be read in at that time.
- ⌘ When a key is pressed, circuitry on the keyboard sends out the ASCII code for the pressed key on eight parallel data line, and then sends out a strobe signal on another line to indicate that valid data is present on the eight data lines.
- ⌘ For higher speed data transfer this method does not work.
- ⌘ The sending system might send data bytes faster than the receiving system could read them. To prevent this handshake data transfer is required.

→ At first valid data is present on data bus and then send strobe the present of data on



put on parallel data-bus signal to indicate data bus.

Figure 6.7: Timing diagram of strobe I/O

3. Single Handshake I/O Data Transfer

- ⌘ The peripheral outputs some parallel data and sends STB signal to MPU.
- ⌘ The MPU detects STB signal on a polled or interrupt basis and reads data byte.
- ⌘ Then the MP sends on ACK signal to the peripheral to indicate that the data has been read and the peripheral can send to next byte of data.

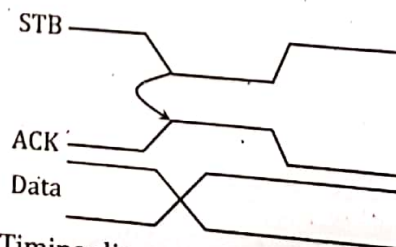


Figure 6.8: Timing diagram of single handshake I/O

4. Double Handshake I/O Data Transfer

- ⌘ For data transfers where coordination is required between sending system and the receiving system, a double handshake is used.
- ⌘ The sending device asserts its STB low to ask "Are you ready?"

- ✓ The receiving system raises its ACK line high to say "I'm ready".
- ✓ The peripheral device then sends the data byte and raises its STB signal high to say "Here is valid data for you".
- ✓ When the receiving system finishes to read the data, receiving system drop its ACK line low to say "I have data than you and I await your request to send the next byte of data".

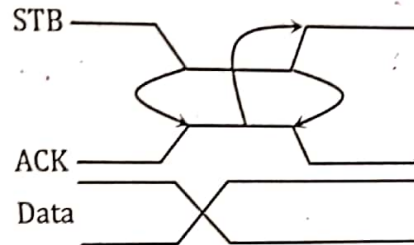


Figure 6.9: Timing diagram of double handshake I/O

6.3 8255A Programmable Peripheral Interface

Introduction

The 8255A PPI provides three 8 bit input/output ports in one 40 pin package. The chip can be interfaced directly to the data bus of the processor allowing its function to be programmed. In one application, a port may appear as an output, but in another by reprogramming it can be as input.

Description

- ✂ The 8255A is a general purpose parallel I/O interfacing device.
- ✂ It provides 24 I/O lines organized as three 8 bit input/output ports labeled A, B and C.
- ✂ Each of the ports A or B can be programmed as an 8 bit input or output port.
- ✂ Port C can be divided in half, with the topmost or bottommost four bits programmed as inputs or outputs.
- ✂ Port C can be used as an 8 bit input or output port as two 4 bit ports or to produce handshake signals for Port A and B.
- ✂ The 8255A is a very versatile device.
- ✂ It can be programmed to look like
 - * Three simple I/O ports (called MODE 0)
 - * Two handshaking I/O ports (called MODE 1)
 - * Port A as a bidirectional I/O port with 5 handshaking signals (called MODE 2) The modes can also be intermixed.
 - * For example, Port A can be programmed to operate in MODE 2 while Port B in MODE 0.

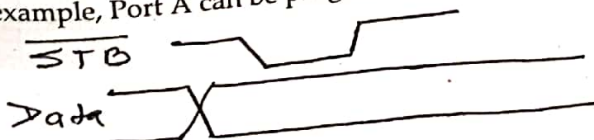


Fig: Strobe I/O.

↳ Data bus buffer
 ↳ Control logic & Group A & Group B Controls

Internal Block Diagram of 8255a

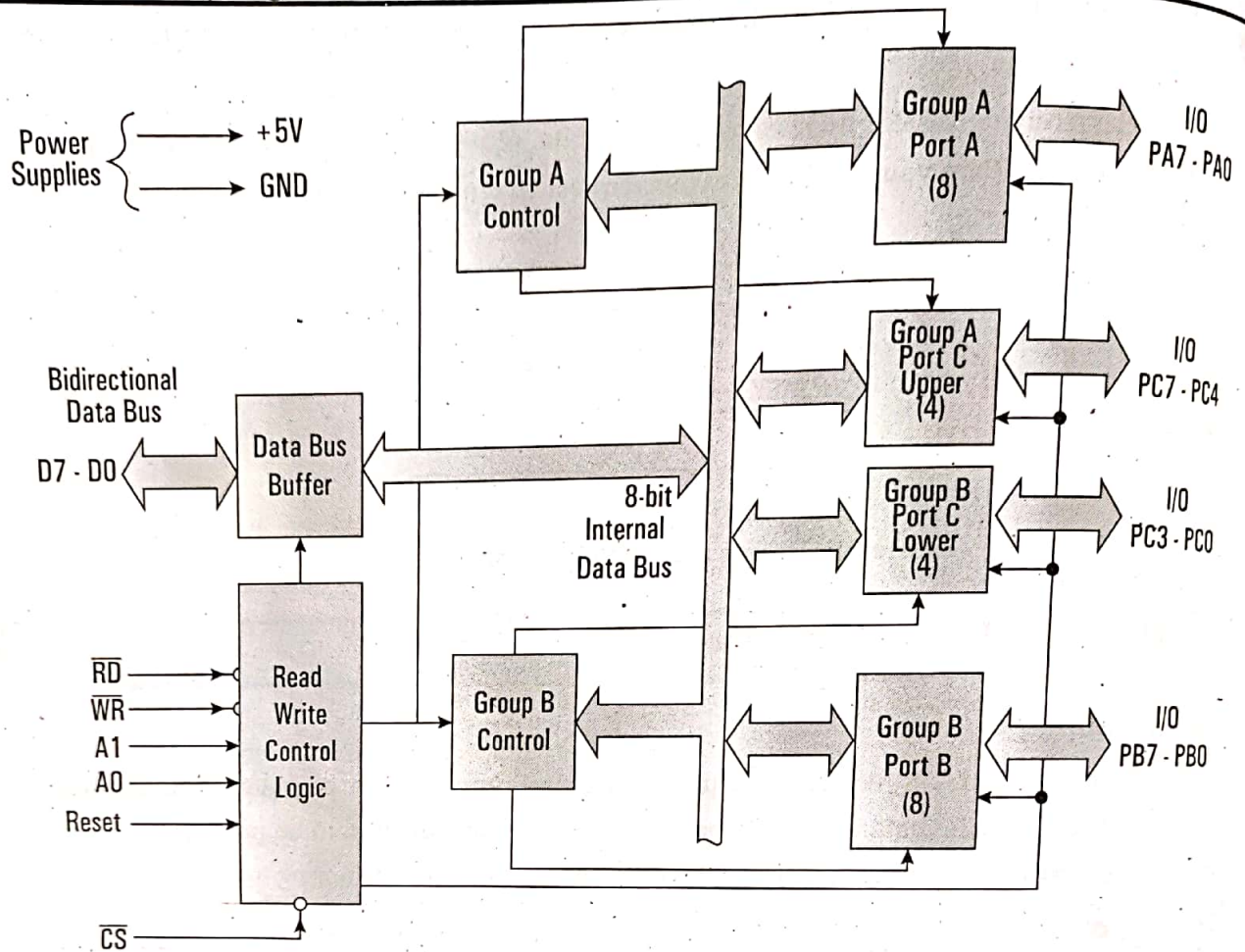


Figure 6.10: Block diagram of 8255 PPI

- ⌘ The address inputs A0, A1 allow you to selectively access one of the three ports or the control register.
- ⌘ The internal addresses for the devices are tabulated below

Description	A1	A0
Port A	0	0
Port B	0	1
Port C	1	0
Control	1	1

- ⌘ The CS input of 8255A enables it for reading or writing
- ⌘ This input is driven by an address decoder.
- ⌘ The RD and WR input pins determine the direction of data flow over the chips 8 bit bidirectional data bus.
- ⌘ The RESET input of 8255A is connected to the system reset line so that, when the system is reset all the port lines are initialized as input lines. This is done to prevent destruction of circuitry connected to port lines.

Control Words

Mode Set Control Word

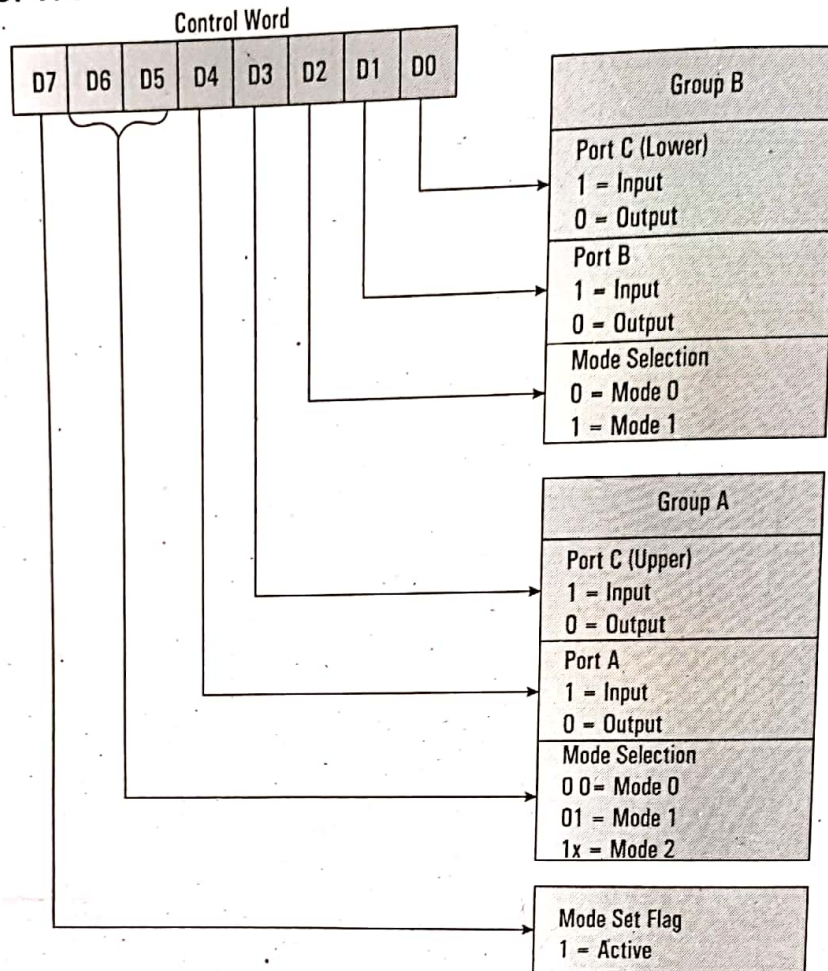


Figure 6.13: Mode Set Control Word

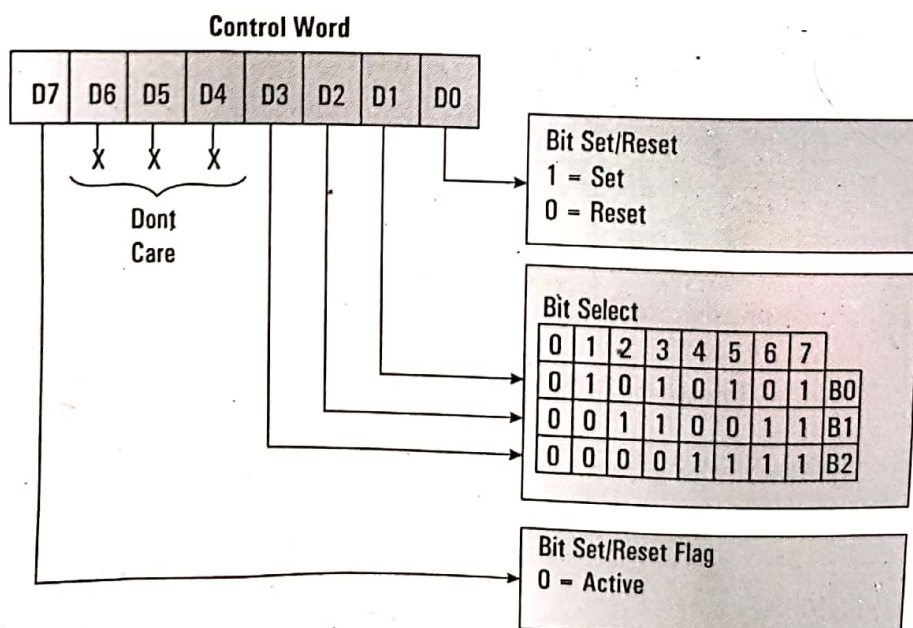
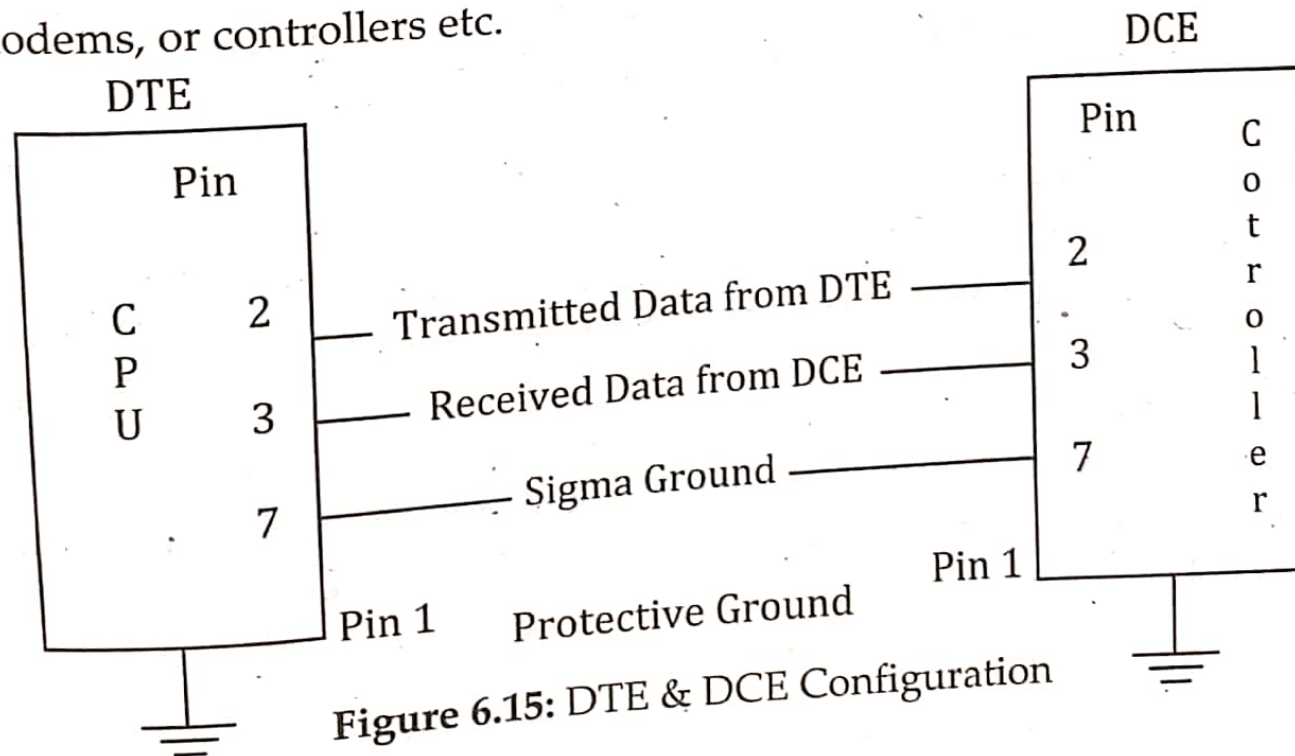
Port C Bit Set/Reset
Control Word

Figure 6.14: Port C Bit Set/Reset

6.4 RS-232 (Recommended Standard-232) Serial Interface

The RS-232(X) is a serial communication protocol, commonly used for transferring and receiving the serial data between two devices. It supports both synchronous and asynchronous data transmissions. Many devices in the industrial environment are still using RS-232 communication cable. Rs-232 cable is used to identify the difference of two signal levels between logic 1 and logic 0. The logic 1 is represented by the -12V and logic 0 is represented the +12V. The RS-232 cable works at different baud rates like 9600 bits/s, 2400 bits/s, 4800 bits/s etc. The RS-232 cable has two terminal devices namely Data Terminal Equipment and Data communication Equipment. Both devices will send and receives the signals. The data terminal equipment is a computer terminal and data communication Equipment is modems, or controllers etc.



Nowadays most of the personal computers have two serial ports and one parallel port (RS232). These two types of ports are used for communicating with external devices and they work in different ways. The parallel port sends and receives the 8-bit data at a time over eight separate wires and this transfers the data very quickly, the parallel ports are typically used to connect a printer to a PC.

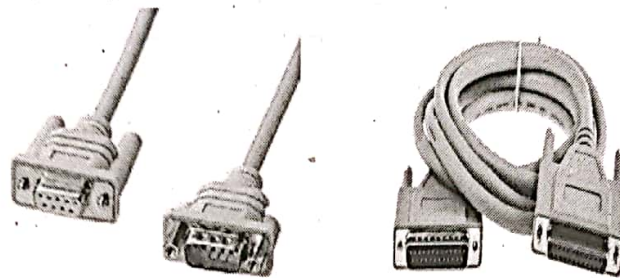


Figure 6.16: RS232 9 pin and 25 pin cables.

A serial port sends and receives one-bit data at a time over one wire and it transfers data very slowly. The RS-232 stands for recommended standard and 232 is a number X indicates the latest version like RS-232c, RS232s.

The most commonly used type of serial cable connectors are 9-pin connectors DB9 and 25-pin connector DB-25. Each of them may be a male or female type. Nowadays most of the computers use the DB9 connector for asynchronous data exchange. The maximum length of RS-232 cable is 50ft.

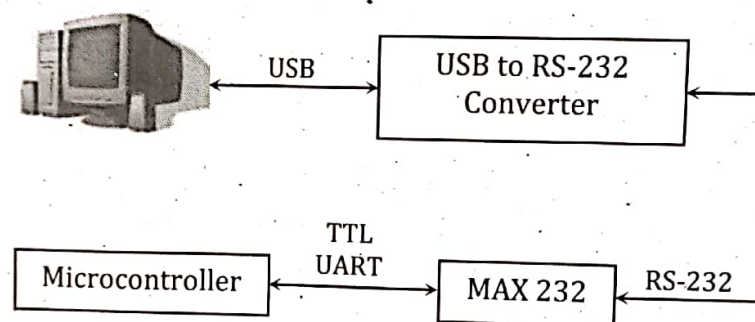


Figure 6.17: RS232 interfacing with computer

RS232 Pin Configuration for 9 Pin

New RS232 has nine pins as mentioned earlier. These nine pins are arranged in the port as shown in RS232 Connector Pinout. The DCE and DTE ports are exactly similar except for the direction of data flow. These nine pins are roughly divided into three categories and we will discuss each category below.

Pin Number	Pin Name	Description
DATA pins (Data flow takes through these pins)		
2	RXD	Receive Data (Data is received through this pin)
3	TXD	Transmit Data (Data is transmitted through this pin)
CONTROL pins (These pins are for establishing interface and to avoid data loss)		
1	CD	Carrier Detect (Set by MODEM when answer is received by remote MODEM)

4	DTR	Data Terminal Ready(Set by PC to prepare MODEM to be connected to telephone circuit)
6	DSR	Data Set Ready(Set by MODEM to tell PC it is ready to receive and send data)
7	RTS	Request To Send(Set by PC to tell MODEM that MODEM can begin sending data)
8	CTS	Clear To send(Set by MODEM to tell PC that it is ready to receive data)
9	RI	Set by MODEM to tell PC a ringing condition has been detected.
REFERENCE		
5	GND	Ground (Used as reference for all pin voltage pulses)

RS232 Features and Specifications

1. RS232 uses Asynchronous communication so no clock is shared between PC and MODEM.
2. Logic '1' on pin is stated by voltage of range '-15V to -3V' and Logic '0' on pin is stated by voltage of range '+3V to +15V'. The logic has wide voltage range giving convenience for user.
3. MAX232 IC can be installed easily to establish RS232 interface with microcontrollers.
4. Full duplex interface of RS232 is very convenient.
5. Two pin simplex RS232 interface can also be established easily if required.
6. A maximum data transfer speed of 19 Kbps(Kilobits per second) is possible through RS232
7. A maximum current of 500mA can be drawn from pins of RS232
8. The interface can be established up to a distance of 50 feet.

Disadvantages of RS232

1. There is no pin dedicated for powering devices (No VCC)
2. More communication pins
3. Switching voltages between +15v and -15v is difficult at higher speeds
4. A maximum speed of 19 Kbps
5. A maximum distance of 50 feet
6. More pins lead to higher noise
7. Only a single device can be connected to RS232 connector unlike I2C
8. Need hardware to convert high voltage logic of RS232 to be compatible to TTL (controller and processor units)

Where RS232 is used?

A few examples where RS232 are suitable to use:

1. When you want a simple communication interface between two units. A two pin full duplex communication can be established easily on RS232 port.
2. RS232 is used in systems where clock sharing is difficult. RS232 is ASYNCHRONOUS so there will be no clock sharing between systems. All you need to do is set data bit rate for each unit.

- Once baud rate is set the units will sample the data according to set baud rate.
- 3. RS232 is also used to control a single unit specifically without delay or errors.
- 4. RS232 interface also delivers data with more accuracy which is a requirement in some cases.

How to use RS232 connector?

As mentioned before we cannot connect RS232 directly to controller we need MAX232 IC to convert high voltage signals to TTL and vice versa. A typical circuit for it is shown below.

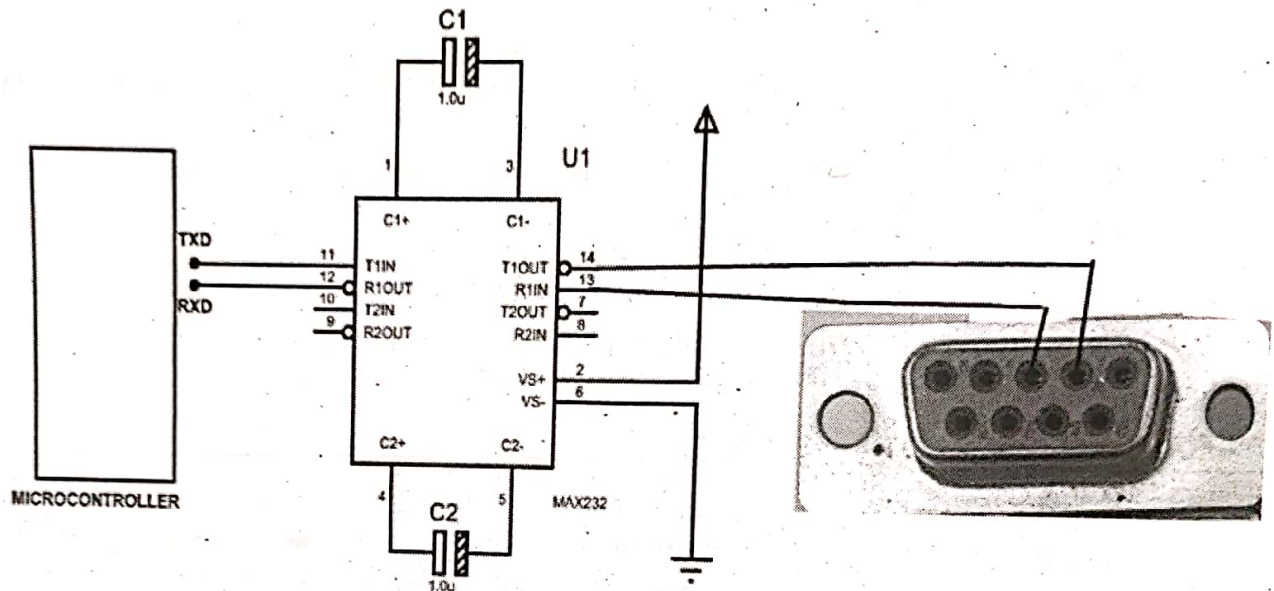


Figure 6.18: RS232 connector

In here we are connecting controller to female RS232 through MAX232 converter chip. The communication voltages reach as high as +15V and as low as -15V in RS232. These voltage levels cannot be used in sensitive electronics so we use a mediator that is MAX232.

The chip converts TTL logic pulses of controller to RS232 voltage level pulses and vice versa. Without this chip you may damage something permanently.

Applications

- ✓ 1. Personal Computers
- ✓ 2. Modems
- ✓ 3. Servers
- ✓ 4. Memory devices
- ✓ 5. Motor control units
- ✓ 6. Printers and Scanners
- ✓ 7. Telephone lines
8. Used basically where serial data transfer is required

RS232 Pin Description for 25 Pin

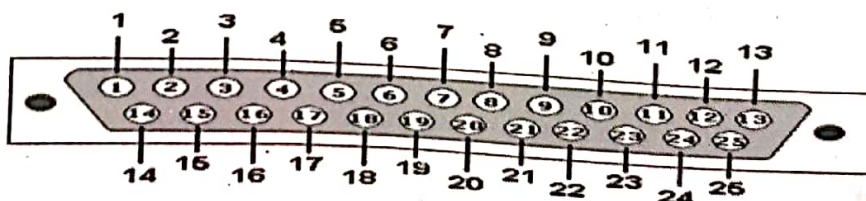


Figure 6.19: 25 Pin RS 232 Connector

It is a 25-pin connector, each pin has its own function is as follows.

PIN 1: (Protective Ground); It is a ground Pin.

PIN 2: Transmit Data.

PIN 3: Receive Data.

PIN 2 & PIN 3: These pins are most important pins for data transmitting and receiving. The 1 & 2-pins are used to data transmission and pin-3 used to data receiving purpose.

PIN 4: Request to send.

PIN 5: Clear to send.

PIN 6: Data Set Ready.

PIN 20: Data terminal Ready.

PIN 4, PIN 5, PIN 6, PIN 20: These pins are the handshaking pins (flow of control). Normally terminals cannot transmit the data until clear to send transmission is received from the DCE.

PIN 7: This pin is the common reference for all signals, including data, timing, and control signals. The DCE and DTE work properly across the serial interface and the pin-7 must be connected both ends without interface would not work.

PIN 8: This pin is also known as received line signal detect or carrier detect. This signal is activated when a suitable carrier is established between the local and remote DCE devices.

PIN 9: This pin is a DTE serial connector, this signal follows the incoming ring to an extent. Normally this signal is used by DCE auto answer mode.

PIN 10: Test Pin.

PIN 11: standby select.

PIN 12: Data Carrier Detect.

PIN 13: Clear to send.

PIN 14: Transmit data.

PIN 15: Transmit clock.

PIN 17: Receive clock.

PIN 24: External Clock.

PIN 15, 17, 24; Synchronous modems use the signals on these pins. These pins are controlled bit timing.

PIN 16: Receive data.

PIN 18: Test Pin.

PIN 19: Request to send.

PIN 21: (Signal Quality Detector); This pin Indicates the quality of the received carrier signal because the transmitting modem must be sent 0 or either 1 at each bit time, the modem controls the timing of the bits from the DTE.

PIN 22: (Ring Indicator): The ringing indicator means the DCE informs the DTE that the phone is ringing. All the modems designed for directly connected to the phone network equipped with auto answer.

PIN 23: Data Signal Rate Detector

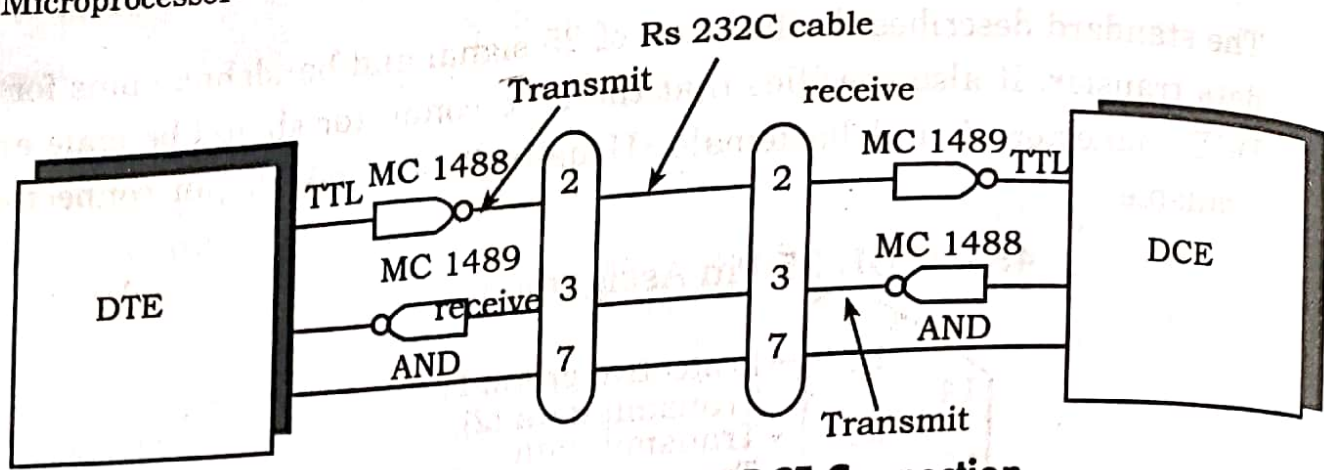


Fig 6.12: RS 232C DTE to DCE Connection

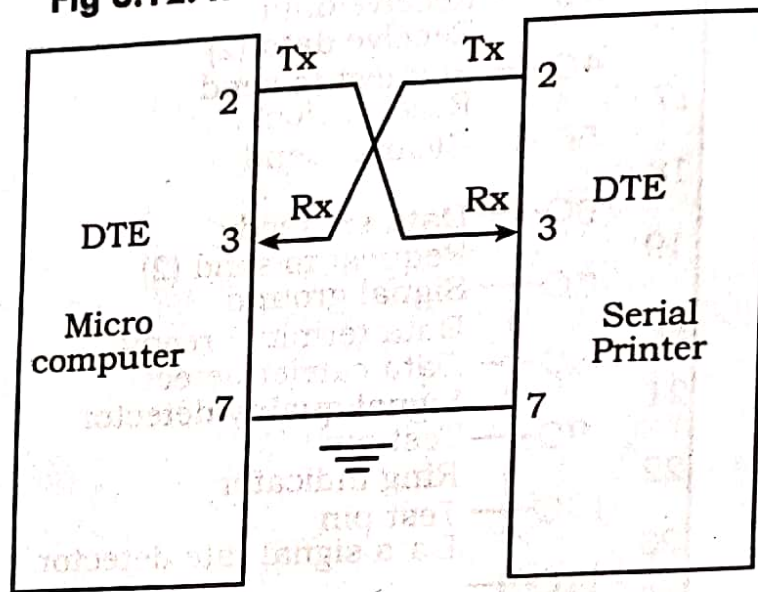


Fig 6.13: 232 DTE to DTE Connection

The signaling in RS-232C is not compatible with the TTL logic level. For TTL 0 v to 0.2 v is considered a logic 0 and 3.4 v to 5v as logic 1. But RS-232C works in a negative logic -3 v to -15v considered as logic 1 and +3 to +15v as logic 0. Because of this incompatibility of data lines with the TTL logic, voltage translators called line drivers and line receivers are required to interface TTL logic with RS-232C signals. The line driver MC 1488 converts logic 1 into approx -9V and logic 0 into +9v. Before it is received by the DCE it is again converted by the line receiver MC 1489 into TTL-Compatible logic. The minimum interfacing required both a computer and a peripheral device requires three lines; pin 2,3 and 7. The lines are defined in relation to the DTE; the terminal transmits on pin 2 and receives on pin 3. On the other hand the DCE transmits on pin 3 and receives on pin 1. Pin 7 is ground pin.